# Analysis and Numerical Implementation of Shooting and Newton-Kantorovich Methods

for a Cubic-Quartic Non-linear Boundary Value Problem

Henry Rochester

MAM3042F - Advanced Numerical Methods

Department of Mathematics and Applied Mathematics
University of Cape Town

March 30, 2026

**Abstract**

We study a cubic-quartic nonlinear boundary value problem and implement two numerical approaches: the shooting method and the Newton-Kantorovich method. These methods are used to compute monotonically decaying and one-node solutions, respectively, of the governing equation. We then determine the range of the parameter $\alpha$ for which these solution branches exist. A comparative analysis of the two methods is carried out, focusing on convergence behaviour, sensitivity to initial guesses, dependence on mesh spacing and length of the spatial interval, as well as overall computational efficiency and ease of implementation.

# Introduction

We consider the non-linear cubic-quartic boundary value problem:

$$\frac{d^2v}{dr^2} + \frac{1}{r}\frac{dv}{dr} - v + 2v^3 - \alpha v^4 = 0, \tag{1}$$

subject to the following boundary conditions:

$$v'(0) = 0, \quad v(\infty) = 0. \tag{2}$$

To compute these solutions, we employ two numerical approaches. The shooting method is used to determine solutions by reformulating the boundary value problem as an initial value problem, while the Newton-Kantorovich method is used to linearise the system and then discretise it using finite differences, allowing us to solve the resulting system numerically.

We begin by analysing the case $\alpha = 0$, which provides a natural reference point and an initial solution from which we can extend the parameter $\alpha$ to determine the range over which the monotonic and one-node solution branches persist.

We then vary the parameter $\alpha$ to investigate how these solution branches change. A diagnostic diagram is constructed to illustrate this dependence. Finally, the performance of the two numerical methods is compared in terms of convergence behaviour, sensitivity to initial guesses, dependence on mesh spacing and the length of the spatial interval, as well as overall computational efficiency and ease of implementation.

# Implementation

## Shooting Method

We begin by transforming the boundary value problem into an initial value problem by introducing a shooting parameter, $p$. Since one boundary condition is given as $v'(0) = 0$, we take the shooting parameter to be the unknown initial value $v(0) = p$.

To solve the second-order ODE numerically, we rewrite it as a system of two first-order equations:

$$\begin{cases} v' = w, \\ w' = -\dfrac{1}{r}w + v - 2v^3 + \alpha v^4, \\ v(0) = p, \\ w(0) = 0. \end{cases}$$

We first consider the case $\alpha = 0$. The goal is to determine the value of $p$ such that the solution satisfies the boundary condition $v(\infty) = 0$.

To compute this value, we use the bisection method. While Newton's method could also be applied, it is highly sensitive to the initial guess. In practice, we found that small deviations in the initial guess caused the method to converge to incorrect solution branches or fail to satisfy the boundary condition at infinity. For this reason, the bisection method provides a more robust approach in this project.

We begin with an initial interval $[1.0, 3.0]$ for $p$, and iteratively bisect this interval. For each candidate value of $p$, we solve the resulting initial value problem using MATLAB's `ode45` solver. Since we cannot evaluate the solution at infinity, we truncate the domain at a finite value $r = L$.

The choice of $L$ is determined empirically, in the following way: we increase $L$ until the computed value of $p$ stabilises. In our implementation, we found that $L = 20$ provides sufficient accuracy. We then compute $v(L)$ and we accept a solution when $|v(L)| < 10^{-8}$. This approximates our boundary condition of $v(\infty) = 0$. Moreover, to ensure termination, we also impose a maximum of 100 bisection iterations. In practice, convergence was achieved well before this limit.

Furthermore, due to the singularity at $r = 0$, we do not initialise exactly at $r = 0$, but instead start the numerical implementation at a small value $\varepsilon = 10^{-6}$.

Using this approach, we obtain both the monotonically decaying and one-node solutions by selecting appropriate initial intervals for the shooting parameter. For the monotonic solution, we use $p \in [1.0, 3.0]$, while for the one-node solution we use $p \in [2.0, 3.0]$. These intervals were found through iterative updates, by observing the qualitative behaviour of the resulting solutions and adjusting the bounds until the desired solution branch was observed.

To determine the range of values of $\alpha$ for which these solution branches exist, we proceed thusly: starting from $\alpha = 0$, we use the corresponding value of $p$ (for each solution

branch) as an initial reference. We then increment $\alpha$ in steps of 0.5, and at each step solve the shooting problem again. If the solver fails, or if the qualitative behaviour of the solution changes (e.g. say the monotonic solution ceases to exist), we reduce the step size for $\alpha$and narrow the interval for $p$ to until the solution branch can no longer be continued or the solver fails. The upper bound and lower bound of $\alpha$ are then recorded.The same procedure is applied for decreasing values of $\alpha$ ($\alpha < 0$), allowing us to approximate the full range over which each solution branch persists.

## Newton-Kantorovich Method

We note that the Newton-Kantorovich method extends the classical Newton root-finding method to a functional setting. The idea here is to linearise the non-linear differential operator and iteratively correct an approximated solution. We proceed as follows:

We begin by defining the non-linear operator:

$$F(v) = v'' + \frac{1}{r}v' - v + 2v^3 - \alpha v^4.$$

To linearise this operator, we compute its Fréchet derivative. We consider expanding the non-linear operator about $v + hz$ as follows:

$$F(v + hz) = (v + hz)'' + \frac{1}{r}(v + hz)' - (v + hz) + 2(v + hz)^3 - \alpha(v + hz)^4$$

Differentiating with respect to $h$ and taking the limit as $h \to 0$, we obtain:

$$F'(v)[z] = z'' + \frac{1}{r}z' - z + 6v^2 z - 4\alpha v^3 z.$$

From this, we obtain the Newton-Kantorovich equation:

$$F'(v)[z] = -F(v),$$

which yields:

$$z'' + \frac{1}{r}z' - z + 6v^2 z - 4\alpha v^3 z = -(v'' + \frac{1}{r}v' - v + 2v^3 - \alpha v^4).$$

We now solve for the correction term $z$, and update the solution via:

$$v_{\text{new}} = v + z.$$

To compute the correction term $z$, we discretise the equation using a finite difference scheme on a mesh-grid $r_1, \ldots, r_N$, with $N$ number of mesh points with uniform spacing. Specifically, we approximate the derivatives as:

$$z''(r_j) \approx \frac{z_{j+1} - 2z_j + z_{j-1}}{h^2}, \qquad z'(r_j) \approx \frac{z_{j+1} - z_{j-1}}{2h}.$$

Now, this yields a linear system of equations, which can be expressed in matrix form:

$$Az = b,$$

where $A$ is the matrix arising from the finite difference discretisation of the linearised operator as per above; $z$ is the vector of unknown corrections, which are trying to solve for and $b = -F(v)$ is the residual vector.

Starting from an initial guess $v$, we solve this system to obtain $z$, and update the solution vector $v$. This process is repeated until the correction satisfies $\|z\|_j < 10^{-8}$ for each $j \in 1, ..., N$ or a maximum of 100 iterations is imposed to ensure termination. We use 100 iterations here to ensure consistent comparison with the Shooting Method. Furthermore, as with the shooting method, we avoid the singularity at $r = 0$ by starting the computation at $\varepsilon = 10^{-6}$.

The boundary condition at infinity is, once again, approximated by truncating the domain at a finite value $r = L = 20$.

For the monotonically decaying solution, we use the initial guess:

$$v(r) = 2e^{-r},$$

while for the one-node solution we use:

$$v(r) = (1 - r^2)e^{-r}.$$

These choices are motivated by the qualitative behaviour of the solutions, namely monotonic decay and the presence of a single node, respectively. In practice, these initial guesses were refined through experimentation.

To determine the range of values of $\alpha$ for which the solutions persist, we again start at $\alpha = 0$, we compute a converged solution and then increment $\alpha$ in steps of 0.5. At each step, the previously computed solution is used as the initial guess for the next iteration. This allows the method to track the solution branch as $\alpha$ varies.

If the method fails to converge, or if the qualitative behaviour of the solution changes (e.g. say loss of monotonicity or number of nodes within solution), we reduce the step size in $\alpha$ to refine the respective upper and lower bound of $\alpha$. The same procedure is applied for decreasing values of $\alpha$ ($\alpha < 0$), allowing us to determine the full range over which each solution branch persists.

# Results

## Solutions for $\alpha = 0$ (Shooting Method)

We begin with the case $\alpha = 0$, which serves as the reference point for the remainder of the investigation. Using the shooting method, with the implementation described above, we determine the values of the shooting parameter $p = v(0)$ that produce solutions satisfying the boundary condition $v(\infty) = 0$.

Using the bisection method to update the shooting parameter, we obtain:

$$p = 1.560931840848$$

for the monotonically decaying solution, and

$$p = 2.356370492159$$

for the one-node solution.

The convergence criterion was taken as:

$$|v(L)| < 10^{-8},$$

which approximates the boundary condition at infinity. Under this condition, the monotonically decaying solution converged in 48 iterations, while the one-node solution converged in 43 iterations. These values were verified across multiple runs.

Figure 1: Monotonically decaying solution with $\alpha = 0$

Figure 2: One-node solution with $\alpha = 0$

To determine suitable intervals for the bisection method, we initially considered the wide interval $[0, 5]$, which produced only multi-node oscillatory solutions. By narrowing the interval, we identified $[1, 3]$ as containing the monotonically decaying branch and

[2, 3] as containing the one-node branch. This shows that although the bisection method itself is robust, the ability to isolate a desired solution branch depends strongly on the interval chosen. A wider interval does not clearly distinguish between qualitatively different behaviours, whereas a narrower interval makes the required branch easier to find. Narrowing the interval did not significantly reduce the number of iterations, but it did improve reliability.

We also implemented Newton's root-finding method to update the shooting parameter. In practice, however, this proved much less robust. Due to its local nature, Newton's method was highly sensitive to the initial guess and, in all cases trialled, produced oscillatory multi-node behaviour rather than the desired monotonic or one-node branches. Even when the initial guesses were chosen close to the correct values, the method often failed to recover the correct branch, which emphasizes its sensitivity compared to bisection.

The truncation endpoint $L$, used to approximate $r \to \infty$, also plays an important role. For moderate values of $L \in [0, 20]$, both desired solution branches were obtained. Increasing $L$ to values such as 30 preserved the qualitative behaviour of the solutions, but the convergence condition $|v(L)| < 10^{-8}$ was no longer satisfied, with values only reaching about $10^{-4}$. For larger values of $L$, numerical instability became more pronounced and the solutions diverged from the desired branches. Thus, larger values of $L$ provide a better formal approximation to infinity, but at the cost of increased instability and sensitivity.

**Range of $\alpha$ Values (Shooting Method)**

Having established the $\alpha = 0$ solutions, we then determined the range of $\alpha$ for which the monotonic and one-node branches persist.

For the monotonically decaying solution, we found an approximate upper bound of

$$\alpha \approx 0.87,$$

beyond which the solution no longer exhibits monotonic decay. On the negative side, we were able to decrease $\alpha$ as far as $-20000$ while still obtaining monotonically decaying behaviour, with $p \approx 0.0759$. This suggests that there may be no finite lower bound, and that the branch may persist as $\alpha \to -\infty$. Based on the computed results, we report the empirical range of:

$$\alpha \in [-20000,\ 0.87].$$

For the one-node solution, the same upper bound,

$$\alpha \approx 0.87,$$

was observed, but the lower bound is much higher, occurring at approximately

$$\alpha \approx -0.9.$$

Hence the one-node branch exists only on the narrower interval

$$\alpha \in [-0.9,\, 0.87].$$

Thus, the monotonic branch persists over a much wider range of $\alpha$ values than the one-node branch.

These results are summarised in the diagnostic plots of $\alpha$ against $v(0) = p$:

Figure 3: Diagnostic Plot for Range of $\alpha$, Monotonically Decaying Solution

Figure 4: Diagnostic Plot for Range of $\alpha$, One-Node Solution

## Solutions for $\alpha = 0$ (Newton-Kantorovich Method)

We now apply the Newton-Kantorovich (NK) method to obtain both the monotonically decaying and one-node solutions.

Using the same convergence criterion as in the shooting method,

$$|v(L)| < 10^{-8},$$

the monotonically decaying solution converged in 5 iterations, while the one-node solution converged in 8 iterations. These values were again consistent across multiple runs. Compared to the shooting method, which required 48 and 43 iterations respectively, this represents a reduction by roughly a factor of 6-8. In terms of iteration count alone, the NK method converges much faster.

However, iteration count is not the only measure of efficiency. Each NK iteration requires the solution of a linear system

$$Az = b,$$

where $A \in \mathbb{R}^{N\times N}$ is the triple diagonal matrix arising from the finite difference discretisation. Since the matrix is tridiagonal, the system can be solved in $\mathcal{O}(N)$ time, while constructing $A$ and $b$ also requires $\mathcal{O}(N)$ operations. Hence the cost per iteration is of order:

$$\mathcal{O}(2N).$$

Thus, although the NK method uses far fewer iterations, each iteration is more computationally involved than in the shooting method.

The mesh resolution also affects the quality of the solution. Initially, we considered $N = 50$, for which the method still converged in 5 and 8 iterations for the two branches. However, the resulting solutions were visibly non-smooth due to the large mesh spacing. As $N$ increased, the solutions became progressively smoother, showing that a finer mesh is necessary to accurately resolve the solution shape. This reflects the usual trade-off between computational cost and accuracy.

The NK method was also found to be less sensitive to the truncation parameter $L$ than the shooting method. We were able to increase $L$ up to 300 while still preserving both the qualitative behaviour of the solutions and the convergence criterion. Even for $L = 500$, the qualitative shape remained unchanged, although the convergence criterion deteriorated to about $10^{-2}$. Hence the NK method is more stable with respect to the choice of spatial interval, even though boundary accuracy degrades for very large $L$.

In contrast, the NK method is highly sensitive to the choice of initial guess. For example, $v(r) = e^{-r}$, although qualitatively similar to the monotonic solution, converged to an incorrect branch. Similarly, $(1 - r)e^{-r}$ failed to produce the correct one-node solution. Through experimentation, we found that:

$$v(r) = 2e^{-r}$$

produces the correct monotonically decaying solution, while:

$$v(r) = 2(1 - r^2)e^{-r}$$

produces the correct one-node solution. Hence the method is sensitive not only to the general shape of the guess, but also to its amplitude and structure. (The solutions obtained are plotted in Figure 1 and Figure 2, respectively).

The NK method is also more involved to implement, since it requires finite difference discretisation, construction of a linear system at each iteration, and careful treatment of the boundary conditions. Nevertheless, this extra complexity is offset by the much faster convergence in iteration count.

### Range of $\alpha$ Values (Newton-Kantorovich Method)

For the monotonically decaying solution, the range of $\alpha$ values obtained using the NK method is very similar to that found using the shooting method. We again obtain the

upper bound:

$$\alpha \approx 0.87,$$

beyond which the branch is no longer monotonically decaying. On the negative side, we were again able to decrease $\alpha$ to $-20000$, suggesting that the lower bound is very likely $-\infty$. Thus the monotonic branch behaves essentially the same under both methods.

For the one-node solution, the main difference is that the positive end of the range is slightly reduced. In this case, the one-node branch was observed over

$$\alpha \in [-0.90,\, 0.75].$$

Thus, while the monotonic branch behaves almost identically under both methods, the one-node branch obtained using the NK method persists over a slightly smaller $\alpha$ interval than in the shooting method. (We plot the diagnostic plots in Figure 3 and 4 respectively).

# Conclusion

In this project, we studied the cubic-quartic non-linear boundary value problem using both the shooting method and the Newton-Kantorovich method. For $\alpha = 0$, both methods successfully recovered the monotonically decaying and one-node solution branches, with closely matching solution shapes.

Overall, the shooting method was simpler to implement and, when combined with bisection, gave robust convergence to the required branches. However, it was more sensitive to the choice of shooting interval and to the truncation length $L$. By contrast, the Newton-Kantorovich method required a more involved implementation, but converged in far fewer iterations and was more stable with respect to the spatial interval. Its main downside was its strong sensitivity to the initial guess.

For non-zero $\alpha$, both methods produced very similar results for the monotonically decaying branch, with an approximate upper bound of $\alpha \approx 0.87$, while no finite lower bound was detected numerically. For the one-node branch, the admissible range of $\alpha$ was much narrower, and the Newton-Kantorovich method produced a slightly smaller interval than the shooting method.

Thus, both methods proved effective for this problem, but with different advantages. The shooting method offered simplicity and robustness, while the Newton-Kantorovich method offered faster convergence and greater stability in spatial interval of length $L$. Comparing the two methods therefore gave useful insight into the practical trade-offs

involved in solving this type of non-linear boundary value problem.

# A MATLAB Implementation of the Shooting Method

Listing 1: MATLAB implementation of the shooting method with bisection update for the parameter $p$.

```
clear; clc; close all;

% params:
L = 20; % we use a finite large approximation for our right Boundary Condition
eps = 1e-6; % since we have a singularity at r=0, we initialise at 1x10^-6
tol = 1e-8; % tolerance for updating shooting param with bisection
maxIter = 100; % maximum number of iterations
alpha = 0.0; % define alpha

% define our first order system of ODEs:
function dYdr = ode(r, Y, alpha)
    v  = Y(1);
    w = Y(2);

    dYdr = zeros(2,1);
    dYdr(1) = w;
    dYdr(2) = -(1/r)*w + v - 2*v^3 + alpha*v^4;
end

% solve ODE and compute the value of v at endpoint:
function v = param_error(p, L, eps, alpha)
    [~, Y] = ode45(@(r,Y) ode(r, Y, alpha), [eps L], [p; 0]);
    v = Y(end,1);
end

% left and right values for shooting param p for bisection method
p_left  = 2.0;
p_right = 3.0;

% Compute residuals at the endpoints
v_left  = param_error(p_left, L, eps, alpha);
v_right = param_error(p_right, L, eps, alpha);

```

```
% use the bisection method to find the value of the shooting param p:
for k = 1:maxIter
    p_mid = 0.5*(p_left + p_right);
    v_mid = param_error(p_mid, L, eps, alpha);

    fprintf('iteration = %d, p = %.12f, v(p) = %.12e\n', k, p_mid, v_mid);

    if abs(v_mid) < tol
        break;
    end

    if v_left * v_mid < 0
        p_right = p_mid;
        v_right = v_mid;
    else
        p_left = p_mid;
        v_left = v_mid;
    end
end

% set the final shooting param:
p_final = p_mid;
fprintf('\nApproximate Final Shooting Parameter: p = %.12f\n', p_final);

% solve the system using our final shooting param to plot:
[r, Y] = ode45(@(r,Y) ode(r, Y, alpha), [eps L], [p_final; 0]);

% get all v values output:
v = Y(:,1);

% Plot solution:
figure;
plot(r, v, 'LineWidth', 2);
xlabel('r');
ylabel('v(r)');
title('Non-linear Shooting Method One-Node Solution with $\alpha = 0$, and $p = 2.356370492159$', 'Interpreter', 'latex');
grid on;
```

# B MATLAB Implementation of the Newton-Kantorovich Method

Listing 2: MATLAB implementation of the Newton-Kantorovich method

```
clear; clc; close all;

% params:
L = 20; % we use a finite large approximation for our right Boundary Condition
eps = 1e-6; % since we have a singularity at r=0, we initialise at 1x10^-6
N = 2000; % number of meshpoints that we use
tol = 1e-8; % tolerance for stopping the update on v
maxIter = 100; % maximum number of iterations
alpha = 0.0;

% setup our mesh grid:
r = linspace(eps, L, N)';
h = L/N;

% initial guess for our v value.
v = 2*exp(-r);
% for one node: v= 2*(1 - r.^2).*exp(-r);

% this is our approximation to v(0):
fprintf('Initial guess: v(0) approx. = %.12f\n', v(1));

for k = 1:maxIter

    % create our system of linear equations in matrix form Az=b
    A = zeros(N, N);
    b = zeros(N, 1);

    % boundary condition at r=0: v'(0)=0
    A(1,1) = -1/h;
    A(1,2) =  1/h;
    b(1)   = -(v(2) - v(1))/h;

    % interior points:
    for j = 2:N-1
        rj = r(j);

```

```
        % finite difference coefficients
        a = 1/h^2 - 1/(2*h*rj); % z_(j-1)
        c = 1/h^2 + 1/(2*h*rj); % z_(j+1)
        z = -2/h^2 - 1 + 6*v(j)^2 - 4*alpha*v(j)^3; % z_j

        A(j,j-1) = a;
        A(j,j)   = z;
        A(j,j+1) = c;

        % residual term
        vdprime = (v(j+1) - 2*v(j) + v(j-1))/h^2;
        vprime  = (v(j+1) - v(j-1))/(2*h);

        Fv = vdprime + (1/rj)*vprime - v(j) + 2*v(j)^3 - alpha*v(j)^4;

        b(j) = -Fv;
    end

    % boundary condition at r = L: v(L)=0
    A(N,N) = 1;
    b(N)   = -v(N);

    % solve linear system
    delta = A \ b;

    % update solution
    v_new = v + delta;

    % convergence check
    err = max(abs(delta));
    fprintf('iteration = %d, max correction = %.12e, v(0) approx = %.12f\n', ...
            k, err, v_new(1));

    v = v_new;

    if err < tol
        break;
    end
end

fprintf('\nApproximate converged value v(0) = %.12f\n', v(1));
```

```
% plot solution
figure;
plot(r, v, 'LineWidth', 2);
xlabel('r');
ylabel('v(r)');
title('Newton-Kantorovich Method with $\alpha = 0$', 'Interpreter', 'latex');
grid on;
```

# References


- N. V. Alexeeva, *MAM3042F: Advanced Numerical Methods Lecture Notes*, Department of Mathematics and Applied Mathematics, University of Cape Town, 2026.
- P. W. Adams, *MAM1043H: Modelling and Applied Computing Lecture Notes*, Department of Mathematics and Applied Mathematics, University of Cape Town, 2024.
- *MATLAB Documentation*, The MathWorks, Inc. Available at: `https://www.mathworks.com/help/` (accessed March 2026).